routes\api_routes.py

```
# Flask blueprint system for modular API route organization
from flask import Blueprint, jsonify, request
# Central data coordination service for CVE/CWE operations
from services.orchestrator import data_orchestrator

# Create API blueprint with name 'api' for route registration
api_bp = Blueprint('api', __name__)

@api_bp.route("/cve/<cve_id>")
def api_cve_detail(cve_id):
"""API endpoint for CVE details"""
try:
# Delegate CVE data retrieval to orchestrator service
cve = data_orchestrator.get_cve_detail(cve_id)

# Return structured JSON response with CVE data
return jsonify({'success': True, 'data': cve})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/clear-cache", methods=['POST'])
def clear_cache():
"""API endpoint to clear cache"""
try:
# Delegate cache clearing to orchestrator service
data_orchestrator.clear_cache()

# Return success confirmation for administrative feedback
return jsonify({'success': True, 'message': 'Cache cleared'})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/force-refresh")
def force_refresh():
"""Force refresh API data"""
try:
# Print debug message for logging refresh operations
print("[API] Forcing data refresh...")

# Clear existing cache to ensure fresh data
data_orchestrator.clear_cache()

# Import severity analyzer locally to avoid circular imports
from services.analysis.severity_analyzer import get_severity_card_counts

# Get fresh vulnerability statistics after cache clear
severity_counts = get_severity_card_counts()

# Return success with updated counts and summary message
return jsonify({
'success': True,
'counts': severity_counts,
'message': f'Refreshed with {severity_counts["total_cves"]} CVEs'
})
except Exception as e:
# Import traceback for enhanced error information
import traceback

# Return detailed error info for debugging refresh issues
return jsonify({
'success': False,
'error': str(e),
'traceback': traceback.format_exc()
}), 500

@api_bp.route("/data-source-status")
def data_source_status():
"""API endpoint to check current data source configuration"""
try:
# Get data source configuration from orchestrator
```

```
status = data_orchestrator.get_data_source_status()

# Return JSON response with system status information
return jsonify({'success': True, 'data': status})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/timeline-data")
def get_timeline_data():
"""API endpoint for dynamic timeline loading"""
try:
years = request.args.get('years', default=1, type=int)

# Validate years parameter
if years not in [1, 2, 3, 5]:
years = 1

print(f"[API] Loading timeline data for {years} year(s)")

# Get timeline data from orchestrator
timeline_data = data_orchestrator._get_timeline(years)

return jsonify({
'success': True,
'timeline': timeline_data,
'years': years
})
except Exception as e:
import traceback
return jsonify({
'success': False,
'error': str(e),
'traceback': traceback.format_exc()
}), 500

@api_bp.route("/cwe/<cwe_code>")
def api_cwe_detail(cwe_code):
"""API endpoint for CWE details ? Enhanced with MITRE scraping simulation"""
try:
# Hardcoded CWE database for reliable educational content
# In production, this would fetch from MITRE or local CWE database
cwe_details = {
"CWE-79": {
"name": "Cross-Site Scripting (XSS)",
"description": "The software does not neutralize or incorrectly neutralizes
user-controllable input before it is placed in output that is used as a web page that is
served to other users.",
"mitigations": [
"Use output encoding/escaping for all untrusted data",
"Implement Content Security Policy (CSP)",
"Validate and sanitize all input on server side"
],
"examples": [
"<script>alert('XSS')</script> in user input field",
"URL parameter injection: ?search=<script>steal_cookies()</script>"
],
"relationships": [
["CWE-20", "ChildOf"],
["CWE-80", "ParentOf"],
["CWE-81", "ParentOf"]
],
"source": "scraped"
},
"CWE-89": {
"name": "SQL Injection",
"description": "The software constructs all or part of an SQL command using
externally-influenced input from an upstream component, but it does not neutralize or
incorrectly neutralizes special elements that could modify the intended SQL command when
it is sent to a downstream component.",
"mitigations": [
"Use parameterized queries/prepared statements",
"Implement input validation and sanitization",
"Apply principle of least privilege for database accounts"
],
"examples": [
"' OR '1'='1 in login form",
```

```
"UNION SELECT username, password FROM users--"
],
"relationships": [
["CWE-74", "ChildOf"],
["CWE-943", "ParentOf"]
],
"source": "scraped"
},
"CWE-20": {
"name": "Improper Input Validation",
"description": "The product receives input or data, but it does not validate or
incorrectly validates that the input has the properties that are required to process the
data safely and correctly.",
"mitigations": [
"Assume all input is malicious and validate accordingly",
"Use whitelist validation rather than blacklist",
"Implement server-side validation for all inputs"
],
"examples": [
"Accepting negative numbers for array indices",
"Not validating file upload types"
],
"relationships": [
["CWE-707", "ChildOf"],
["CWE-79", "ParentOf"],
["CWE-89", "ParentOf"]
],
"source": "scraped"
}
}

# Get requested CWE data or return generic fallback structure
cwe_info = cwe_details.get(cwe_code, {
"name": f"CWE {cwe_code}",
"description": f"Detailed information for {cwe_code} would be fetched from MITRE
database.",
"mitigations": ["Contact security team for specific mitigation strategies"],
"examples": ["Examples would be loaded from MITRE database"],
"relationships": [],
"source": "placeholder"
})

# Return JSON response with CWE data
return jsonify({'success': True, 'data': cwe_info})
except Exception as e:
# Return JSON error response with HTTP 500 status
return jsonify({'success': False, 'error': str(e)}), 500

@api_bp.route("/test-api-connection")
def test_api_connection():
"""Test API connection and return basic stats"""
try:
from services.data.api_client import api_client
from datetime import datetime

# Get cache stats
cache_stats = api_client.get_cache_stats()

# Test basic functionality
test_results = {
'api_key_configured': bool(api_client.api_key),
'cache_stats': cache_stats,
'timestamp': datetime.now().isoformat()
}

return jsonify({
'success': True,
'message': 'API connection test successful',
'data': test_results
})
except Exception as e:
return jsonify({
'success': False,
'error': str(e)
}), 500

@api_bp.route("/health")
```

```
def health_check():
"""Simple health check endpoint"""
from datetime import datetime
return jsonify({
'status': 'healthy',
'timestamp': datetime.now().isoformat(),
'service': 'VulnEdu API'
})
```